

QuakeFloat8: A Provably Optimal Log-Domain 8-Bit Number Format

April 2026

Noah Everett

Department of Physics, Harvard University

Abstract

QuakeFloat8 (QF8) is a block-scaled logarithmic 8-bit number format in which multiplication reduces to 7-bit integer addition. The format uses a 1-bit sign, a 7-bit `u3.4` fixed-point $\log_2$ code per element, and a shared E8M0 block exponent per 32 elements.

We prove that QF8’s log-uniform spacing is minimax-optimal: the *equalization property* guarantees that the mean squared relative error (MSRE) equals $\varepsilon^2/12$ for *any* source distribution, and no other fixed-rate quantizer achieves lower worst-case normalized MSE (Theorem 4.4). This density-independence eliminates the need for per-signal calibration.

Gate-level estimates show the QF8 multiply path at ~ 66 gates vs. ~ 231 for FP8 ($3.5\times$ cheaper), with the full MAC unit at ~ 650 gates vs. ~ 1350 for FP8 ($\sim 2\times$ cheaper). All estimates are NAND2-equivalent; no RTL synthesis has been performed.

Empirical validation across 1,481 weight tensors from 15 pretrained models confirms the equalization property: QF8’s SQNR is 38.1 ± 0.10 dB — a +6.6 dB advantage over FP8 E4M3 at identical storage cost. Weight-only post-training quantization from 124M to 7B parameters shows degradation $\leq 0.03\%$ on WikiText-2 perplexity. The format’s theoretical guarantees — distribution-independent quantization error, multiplication via integer addition, and low hardware cost — apply broadly to any domain requiring efficient fixed-point arithmetic with controlled relative error, including telecommunications, edge computing, and scientific computing.

Contents

1	Introduction	3
1.1	The 8-Bit Frontier	3
1.2	The Quake Insight	3
1.3	Prior Work	3
1.4	Contributions	4
2	The QF8 Format	4
2.1	Encoding Scheme	4
2.2	Logarithmic Spacing	5
2.3	Multiplication as Integer Addition	5
2.4	Accumulation Strategy	6
2.5	BMD Realizability	6
2.6	Format Comparison	6
3	Hardware Cost Analysis	6
3.1	Reference Data	6
3.2	QF8 Multiply Path	7
3.3	QF8-Medium MAC Unit	7

3.4	Master Comparison	7
3.5	Systolic Array Scaling (128×128)	7
3.6	Break-Even Analysis	7
3.7	Summary	8
4	The Equalization Property	9
4.1	Notation	9
4.2	Main Results	9
4.3	Relation to Classical Theory	11
4.4	Numerical Verification	11
5	Empirical Validation	11
5.1	SQNR on Pretrained Weights	12
5.2	Cross-Model Equalization Experiment	12
5.3	Post-Training Quantization on WikiText-2	13
5.3.1	Scaling to 7B Parameters	14
5.4	Quantization-Aware Training	16
5.5	Synthetic Benchmarks	16
6	Future Work	17
6.1	Remaining Gaps	17
6.2	Gradient Quantization	18
6.3	BMD Decoder Conjecture	18
A	Product Error Decomposition	18
A.1	Auxiliary Quantities	18
A.2	The General Formula	18
A.3	Centroid Simplification	19
A.4	Specialization to Log-Uniform Quantizers	19
A.5	Optimality for Multiplication	20
A.6	Extension to Dot Products	20
B	Exact MSRE Derivation	20
C	Exponential Separation and Format Comparison	21
C.1	Log vs. Uniform: Exponential MSRE Separation	21
C.2	QF8 vs. FP8 E4M3: Levels-per-Octave Advantage	22
C.3	Comparison with Johnson’s ELMA	22
D	Rate-Distortion Context	23
D.1	Gaussian Source Rate-Distortion	23
D.2	Scalar Quantization Gap (Zador)	23
D.3	Bit Allocation	23

Introduction

The 8-Bit Frontier

The push toward low-precision arithmetic has been remarkably successful across multiple domains. In machine learning, FP16 mixed-precision training [1] is standard practice, BFloat16 is ubiquitous, and 8-bit formats (INT8, FP8 E4M3/E5M2 [2]) are deployed in production hardware (NVIDIA H100, AMD MI300, Google TPU). The OCP Microscaling (MX) specification [3] codifies 8 bits per element with block scaling as sufficient for both training and inference of large language models. In telecommunications, logarithmic companding (μ -law and A-law) has been the foundation of PCM quantization since the 1960s [4], providing approximately constant signal-to-noise ratio across signal levels. In edge computing and IoT, the energy cost of arithmetic — particularly multiplication, which dominates at $> 10\times$ the cost of addition [5] — makes low-precision formats with cheap multiply operations attractive for on-device inference. In scientific computing, reduced-precision formats accelerate simulations in climate modeling, molecular dynamics, and computational physics, where *relative* accuracy across many orders of magnitude matters more than absolute precision near zero.

Yet a gap remains between the precision delivered by current 8-bit formats and what is theoretically achievable. FP8 E4M3 provides 8 representable levels per octave (factor of 2). Is this optimal? Can we do better at the same bit budget?

The Quake Insight

In 1999, the Quake III source code revealed a remarkable hack: the integer bit pattern of an IEEE 754 float is approximately a linear function of $\log_2(x)$. Reinterpreting float bits as integers, and vice versa, allows cheap approximate logarithmic arithmetic.

QF8 makes this accident intentional. By storing values as fixed-point $\log_2$ codes, multiplication becomes integer addition — replacing an $O(n^2)$ -gate multiplier with an $O(n)$ -gate adder. The format is paired with MX-style block scaling (E8M0 shared exponent per 32 elements) to achieve full FP32 dynamic range.

Prior Work

Log-domain ML arithmetic. The closest prior work is Johnson’s (2018) ELMA format [6], which demonstrated that log-domain multiply with Kulisch accumulation is viable for 8-bit deep learning — QF8’s core multiply-as-addition mechanism is Johnson’s. Johnson used a posit-style tapered encoding [7], which provides variable precision across the range; QF8 replaces this with a simpler fixed-width u3.4 code. Miyashita et al. [8] explored logarithmic data representation for CNNs, and the Logarithmic Number System (LNS) has been studied since the 1970s [9, 10].

Block scaling. The OCP Microscaling (MX) specification [3] established E8M0 block scaling as a practical way to decouple dynamic range from per-element precision. QF8 adopts this directly: the difference is the per-element encoding (log code vs. IEEE-like floating point).

Distribution-aware quantization. NF4 [11] (QLoRA) places levels at Gaussian quantiles, achieving optimal quantization for a specific distribution. QF8’s log-uniform spacing takes the opposite approach: minimax optimality across *all* distributions (Section 4).

Classical quantization theory. Bennett [12] established the high-resolution framework for quantization distortion. Panter and Dite [13] derived the MSE-optimal compander (the one-third-power rule), which depends critically on the source distribution. Smith [4] observed that logarithmic companding achieves approximately amplitude-independent SNR for PCM telephony, but his result concerns signal level, not source distribution, and his distortion formula remains distribution-dependent (e.g., 0.707 for exponential, 0.798 for Gaussian inputs). Most directly related, Sun and Goyal [14] proved that logarithmic companding is *variable-rate* optimal for MSRE but showed it is *not* generally optimal for fixed-rate quantization. Theorem 4.4 provides the complementary fixed-rate minimax result: the equalization property makes log-uniform quantization uniquely density-independent, resolving the fixed-rate question. The standard references [15, 16] provide comprehensive treatments of quantization and companding theory.

Contributions

1. **The equalization property and minimax optimality.** We prove that log-uniform quantization achieves $\text{MSRE} = \varepsilon^2/12$ for *any* source density (Lemma 4.3), because the x^2 terms in the NMSE numerator and denominator cancel exactly. This density-independence implies that log-uniform quantization is the unique fixed-rate minimax-optimal quantizer for NMSE over all source distributions (Theorem 4.4), complementing Sun and Goyal’s [14] variable-rate result with a fixed-rate minimax guarantee.
2. **Format design.** QF8 combines a u3.4 fixed-point log code (16 levels per octave) with E8M0 block scaling, achieving $2\times$ the precision per octave of FP8 E4M3 at the same effective storage cost (8.25 bits/element).
3. **Hardware analysis.** Gate-level estimates (NAND2-equivalent, not synthesized) show the QF8 multiply path at ~ 66 gates vs. ~ 231 for FP8 ($3.5\times$ cheaper). The full MAC unit is estimated at ~ 650 gates vs. ~ 1350 for FP8 ($\sim 2\times$ cheaper), achieving near-INT8 area while providing floating-point dynamic range. The $3.5\times$ multiply-path advantage is partially offset by the accumulation overhead (LUT decode + barrel shifter + Kulisch register), yielding the $\sim 2\times$ net advantage for the complete MAC.
4. **Empirical validation.** The equalization property is confirmed across 1,481 weight tensors from 15 models and 5 architectures: QF8’s SQNR is 38.1 ± 0.10 dB regardless of model or weight distribution. Weight-only PTQ from GPT-2 (124M) to Mistral-7B shows degradation $\leq 0.03\%$ at 7B scale, vs. MXFP8’s $+0.19\text{--}0.28\%$. W8A8 quantization exposes a limitation: QF8’s log-uniform spacing is suboptimal for activations concentrated near zero.

The QF8 Format

Encoding Scheme

Definition 2.1 (QF8 Encoding). A QF8-encoded block of $k = 32$ values consists of:

- **Shared block scale** (8 bits): An E8M0 power-of-2 exponent $s \in \{2^{-127}, \dots, 2^{127}\}$.
- **Per-element code** (8 bits each): 1 sign bit $\sigma \in \{0, 1\}$ and a 7-bit unsigned u3.4 fixed-point code $c \in \{0, 1, \dots, 127\}$.

The reconstruction formula is:

$$\hat{x} = (-1)^\sigma \cdot s \cdot 2^{(c-64)/16}, \quad c \neq 0$$

with $c = 0$ encoding exact zero. The effective storage cost is $8 + 8/32 = 8.25$ bits per element.

Logarithmic Spacing

The `u3.4` code provides 3 integer bits and 4 fractional bits of $\log_2 |x/s|$, spanning 8 octaves with *16 representable levels per octave*. This is the *log-uniform quantizer*: the ratio between consecutive representable values is constant at $r = 2^{1/16} \approx 1.0443$. With geometric midpoints, the maximum relative quantization error is

$$\sqrt{r} - 1 = 2^{1/32} - 1 \approx 2.2\%.$$

Compare FP8 E4M3, which has 8 levels per octave (3-bit mantissa) and a maximum relative error of $1/16 = 6.25\%$ within each binade.

Remark 2.2 (Bit Allocation). The 7-bit log code admits several integer/fractional splits. The `u3.4` choice balances range and precision:

- **Per-element range:** 3 integer bits give $2^3 = 8$ octaves. With E8M0 block scaling absorbing inter-block variation, the per-element code only needs to cover *within-block* spread. For typical signal blocks of size 32 (e.g., ML weight tensors, audio frames), empirical within-block spread is 3–5 octaves; 8 octaves provides comfortable margin.
- **Precision:** 4 fractional bits give 16 levels per octave (MSRE = 1.56×10^{-4}). The alternative `u4.3` (16 octaves, 8 levels/octave) halves precision for already-sufficient range. The alternative `u2.5` (4 octaves, 32 levels/octave) risks overflow for blocks with > 4 octave spread.
- **Wide-range variant:** For signals requiring wider dynamic range (e.g., gradients in ML training), a `u4.3` variant (16 octaves, 8 levels/octave) is the natural analog of FP8’s E5M2 (Section 6).

Multiplication as Integer Addition

For two QF8 values with log codes c_a, c_b :

$$\text{sign}_{\text{product}} = \sigma_a \oplus \sigma_b \tag{1}$$

$$\text{log-code}_{\text{product}} = c_a + c_b \tag{2}$$

The product’s log code is a simple 7-bit integer addition. In hardware, this replaces the 4×4 mantissa multiplier needed for FP8 E4M3 with a 7-bit carry-lookahead adder — an estimated $\sim 3.5\times$ reduction in multiply-path gate count (66 vs. 231 gates). The full MAC unit including accumulation is estimated at $\sim 2\times$ cheaper (650 vs. 1350 gates; see Section 3). The difference between the $3.5\times$ multiply-path advantage and the $\sim 2\times$ full-MAC advantage arises from the accumulation stage: QF8 requires a lookup table and barrel shifter to convert products back to linear domain before accumulation (Section 2.4), partially offsetting the multiply-path savings.

Accumulation Strategy

The classical weakness of log-domain arithmetic is that *addition* in the original domain is expensive. QF8 adopts the *Kulisch accumulation* strategy [6]:

1. Multiply in log domain (integer addition of codes).
2. Convert each product to linear domain via a $2^4 = 16$ -entry lookup table.
3. Accumulate in a wide fixed-point register (32–48 bits).

This hybrid log-multiply / linear-accumulate approach gets cheap multiplies *and* exact (or near-exact) accumulation.

BMD Realizability

Definition 2.3 (Bit-Manipulation Decodable (BMD)). A decoder $\hat{D} : \{0,1\}^B \rightarrow \mathbb{R}$ is *bit-manipulation decodable* if it can be expressed as a fixed-length straight-line program over $\{+, -, \times, \gg, \ll, \&, |, \oplus\}$ on machine-word integers, with the output reinterpreted as a float via type punning.

Proposition 2.4 (BMD Realizability of QF8). *The log-uniform decode function $\hat{D}(n) = a \cdot r^n$ admits three BMD implementations:*

1. **Lookup table:** $O(N)$ storage, $O(1)$ time.
2. **Reinterpret cast (Quake trick):** $I = \alpha n + \beta$, then float reinterpret. Cost: 1 multiply + 1 add + type pun = 3 ops.
3. **Quadratic approximation:** $2^{e+f} = 2^e \cdot (1 + 0.6602f + 0.3398f^2)$. Cost: 2 multiplies + 2 adds + 1 shift = 5 ops. Maximum relative error: 0.27%.

Proof. Each construction can be verified by direct calculation. The LUT is immediate. The reinterpret cast follows from the observation that an IEEE 754 float’s bit pattern $I = 2^{23}(e+127)+m$ is affine in $\log_2(x)$. The quadratic approximation is a minimax-optimal polynomial fit to 2^f on $[0, 1)$. $\square$

Format Comparison

Hardware Cost Analysis

This section presents gate-level hardware models anchored to published silicon data from Horowitz (ISSCC 2014) [5], Johnson (2018 ELMA ASIC at 28nm) [6], and NVIDIA architecture whitepapers. *All gate counts are NAND2-equivalent estimates.* No RTL has been written and no synthesis has been performed. The numbers should be treated as order-of-magnitude guidance, not as measured silicon results.

Reference Data

Horowitz ISSCC 2014 (45nm CMOS): 8-bit integer multiply = 0.2 pJ, 8-bit add = 0.03 pJ, 32-bit FP multiply = 3.7 pJ.

Johnson ELMA synthesis (28nm): 8-bit ELMA/38-bit Kulisch = $0.96\times$ power, $1.12\times$ area vs. INT8/32-bit MAC. 16-bit ELMA = $0.59\times$ power, $0.68\times$ area vs. FP16 FMA.

Table 1: Comparison of 8-bit and related number formats. All block-scaled formats use $k = 32$ elements per block. Gate counts are NAND2-equivalent estimates.

Feature	INT8	FP8 E4M3	FP8 E5M2	BFloat16	QF8
Per-element bits	8	8	8	16	8
Block scaling	sym	E8M0	E8M0	—	E8M0
Eff. bits/element	8.25	8.25	8.25	16	8.25
Levels per octave	varies	8	4	128	16
Multiply gates	400	231	231	~850	66
Full MAC gates	750	1,350	1,350	2,400	650
SQNR $\mathcal{N}(0, 1)$	44.6*	31.5	~25	~52	38.1
Calibration-free	No	No	No	—	Yes[†]

*INT8 optimizes MSE, not NMSE; SQNR is distribution-dependent.

[†]For weight quantization. Activation quantization requires calibration (Section 5.3).

QF8 Multiply Path

The core QF8 advantage: multiplication reduces to a 7-bit integer addition plus sign XOR.

Table 2: QF8 multiply-path gate count.

Component	Gates	Area (μm^2 , 7nm)
Sign XOR	1	0.04
7-bit CLA adder	50	2.0
Overflow/saturation detect	5	0.2
Zero detection	10	0.4
Total: multiply	66	2.6

Compare: INT8 8×8 multiplier = 400 gates; FP8 4×4 mantissa multiplier + exponent logic = 230 gates; FP16 11×11 multiplier = 850 gates; FP32 24×24 multiplier = 3200 gates. The QF8 multiply path is $3.5\times$ to $48\times$ cheaper than alternatives.

QF8-Medium MAC Unit

The recommended design point: 6-bit barrel shifter and 32-bit carry-save Kulisch accumulator for intra-block accumulation, with block-pair results reduced via FP32 arithmetic (matching MX-format accelerator practice).

Master Comparison

Systolic Array Scaling (128×128)

Break-Even Analysis

The log-domain advantage scales differently with bit-width because the multiplier is $O(n^2)$ while the adder is $O(n)$:

The crossover is at ~ 7 – 8 bits. Below 7 bits, the LUT and accumulator overhead exceeds multiplier savings. Above 8 bits, log-domain wins decisively and the advantage grows quadratically.

Table 3: QF8-Medium MAC unit breakdown (7nm, 1 GHz).

Component	Gates	Area (μm^2)
Multiply path (Table 2)	66	2.6
exp2 LUT (16×12 -bit ROM)	100	4.0
6-bit barrel shifter	150	6.0
32-bit carry-save Kulisch accumulator	300	12.0
Control/mux	34	1.4
Total	650	26.0

Table 4: Per-MAC unit comparison at 7nm, 1 GHz.

Metric	INT8	FP8 E4M3	FP16	FP32	QF8-Med
Gate count	750	1,350	2,400	5,950	650
Area (μm^2)	30	54	96	238	26
Energy (pJ/MAC)	0.040	0.070	0.130	0.350	0.033
Pipeline stages	2	3	3	3–4	2
Critical path (gates)	24–30	35–45	38–48	55–70	20–26

This confirms Johnson’s [6] finding that 16-bit ELMA achieves 41% power reduction over FP16.

Summary

- **QF8 vs FP32/FP16:** Clear, large, robust win (3–10 $\times$ savings).
- **QF8 vs FP8:** Clear win ($\sim 2\times$ area and power).
- **QF8 vs INT8:** Slight win ($0.87\times$ area, $0.83\times$ power). The carry-save Kulisch design tips the balance.
- **The real value:** $2\times$ precision-per-octave at INT8-equivalent hardware cost and half the cost of FP8.

System-level efficiency. The per-MAC energy advantages above apply to the compute datapath only. Modern inference workloads are typically memory-bandwidth-bound, where data-movement energy dominates arithmetic cost by two orders of magnitude [5]. In this regime, QF8’s primary efficiency contribution is indirect: at identical 8-bit bandwidth, QF8 delivers +6.6 dB higher SQNR than FP8 E4M3, potentially enabling 8-bit deployment in settings that would otherwise require 16-bit precision — halving memory traffic and the associated energy cost. Validating this claim requires demonstrating QF8 at 8 bits matching FP16 quality on production-scale models, which we leave to future work.

Table 5: QF8-Medium savings ratios vs. each baseline.

Metric	vs FP32	vs FP16	vs FP8	vs INT8
Area savings	89%	73%	52%	13%
Power savings	91%	75%	53%	17%
Latency improvement	55–65%	35–45%	25–35%	~5–10%

Table 6: 128×128 systolic array estimates at 7nm, 1 GHz.

Metric	INT8	FP8	FP16	FP32	QF8-Med
Compute area (mm ²)	0.49	0.88	1.57	3.90	0.43
Total power (W)	0.68	1.19	2.21	5.93	0.56
TOPS/W	48.2	27.6	14.8	5.5	58.5
TOPS/mm ²	66.9	37.3	20.9	8.4	76.2

The Equalization Property

Notation

Let X be a positive random variable with probability density function (pdf) f supported on $[a, b] \subset \mathbb{R}_{>0}$. A B -bit scalar quantizer is a map $Q : [a, b] \rightarrow \hat{\mathcal{X}}$ with $N = 2^B$ reconstruction points partitioning $[a, b]$ into cells $\{S_k\}_{k=0}^{N-1}$ with codepoints $\{c_k\}$.

Definition 4.1 (Distortion Measures). Let $\delta_X = X - Q(X)$ and $\varepsilon_X = \delta_X/X$.

$$\text{MSE}_X = \mathbb{E}[\delta_X^2], \quad \text{NMSE}_X = \frac{\mathbb{E}[\delta_X^2]}{\mathbb{E}[X^2]} \quad (3)$$

$$\text{MSRE}_X = \mathbb{E}[\varepsilon_X^2], \quad \text{SQNR} = 10 \log_{10}(1/\text{NMSE}) \text{ dB} \quad (4)$$

Definition 4.2 (Log-Uniform Quantizer). The log-uniform quantizer with N levels over dynamic range $[a, b]$ has cell boundaries $a \cdot r^k$ for $k = 0, \dots, N$, where $r = (b/a)^{1/N}$ and $R = \log_2(b/a)$ is the range in octaves. With geometric midpoint codepoints $c_k = a \cdot r^{k+1/2}$, the relative cell width is constant:

$$\frac{w(x)}{x} = r - 1 = e^\varepsilon - 1 \approx \varepsilon, \quad \varepsilon = \frac{R \ln 2}{N} \quad (5)$$

Main Results

Lemma 4.3 (Equalization Property). *For the log-uniform quantizer in the high-resolution regime ($N \gg 1$):*

$$\text{MSRE} = \frac{\varepsilon^2}{12} + O(\varepsilon^4)$$

for any pdf f supported on $[a, b]$. The MSRE is density-independent.

Proof. For the log-uniform quantizer, $w(x) = x \cdot \varepsilon + O(x\varepsilon^2)$. Under Bennett’s [12] high-resolution integral approximation:

$$\text{NMSE} = \frac{\int_a^b \frac{w(x)^2}{12} f(x) dx}{\int_a^b x^2 f(x) dx} = \frac{\frac{\varepsilon^2}{12} \int_a^b x^2 f(x) dx}{\int_a^b x^2 f(x) dx} = \frac{\varepsilon^2}{12}$$

Table 7: Log-domain break-even across bit widths.

Bit-width	Mult saved (gates)	LUT overhead (gates)	Net	vs INT- N
4	38	120	−82	1.5× worse
6	130	180	−50	1.2× worse
8	364	300	+64	0.92× (slight win)
12	1,145	500	+645	0.63×
16	2,925	800	+2,125	0.40×

The x^2 terms in the numerator and denominator cancel exactly. Similarly, $\text{MSRE} = \varepsilon^2/12$ for any density f . $\square$

Theorem 4.4 (Minimax NMSE Optimality of Log-Uniform Quantization). *In the high-resolution regime ($N \gg 1$), among all N -level quantizers on $[a, b]$, the log-uniform quantizer uniquely minimizes the worst-case NMSE (and MSRE) over all densities:*

$$Q_{\log}^* = \arg \min_{Q \in \mathcal{Q}_N} \max_{f \in \mathcal{F}[a,b]} \text{NMSE}(Q, f)$$

The minimax value is:

$$\text{NMSE}^* = \frac{\varepsilon^2}{12} + O(\varepsilon^4) = \frac{(R \ln 2)^2}{12N^2} \quad (6)$$

For $B = 8$ bits, $R = 16$ octaves: $\text{NMSE}^* = 1.564 \times 10^{-4}$ (SQNR = 38.1 dB).

Proof. By Lemma 4.3, the log-uniform quantizer achieves $\text{NMSE} = \varepsilon^2/12$ for every density f . It remains to show that any non-log-uniform quantizer has strictly higher worst-case NMSE.

Step 1 (Coverage constraint). Define $\phi(x) = w(x)/x$ (relative cell width). Any N -cell quantizer on $[a, b]$ satisfies:

$$\int_a^b \frac{dx}{\phi(x)x} = N \quad (7)$$

For the log-uniform quantizer, $\phi \equiv \varepsilon$, giving $(1/\varepsilon) \ln(b/a) = N$. Suppose for contradiction that ϕ is non-constant but $\phi(x) \leq \varepsilon$ everywhere. Then $1/\phi(x) \geq 1/\varepsilon$ everywhere, with strict inequality on a set of positive measure, so

$$N = \int_a^b \frac{dx}{\phi(x)x} > \int_a^b \frac{dx}{\varepsilon x} = \frac{\ln(b/a)}{\varepsilon} = N,$$

a contradiction. Therefore $\phi^* = \max_x \phi(x) > \varepsilon$ for any non-log-uniform quantizer.

Step 2 (Adversarial density). Since ϕ is piecewise constant on finitely many cells, the maximum ϕ^* is achieved in the interior of some cell S_{k^*} . For any $\eta > 0$ small enough that $[x^* - \eta, x^* + \eta] \subset S_{k^*}$, the uniform density on $[x^* - \eta, x^* + \eta]$ achieves:

$$\text{MSRE} \geq \frac{(\phi^*)^2}{12} - g(\eta), \quad g(\eta) \rightarrow 0 \text{ as } \eta \rightarrow 0$$

Therefore $\max_f \text{MSRE}(Q, f) \geq (\phi^*)^2/12 > \varepsilon^2/12 = \text{MSRE}^*$.

Step 3 (Uniqueness). If $Q \neq Q_{\log}$, then ϕ is non-constant and Steps 1–2 give strict inequality. $\square$

Corollary 4.5 (Minimax MSRE). *The same quantizer also minimizes worst-case MSRE. The proof is identical, with $w(x)^2 f(x)/x^2$ replacing $w(x)^2 f(x)/\mathbb{E}[X^2]$.*

Remark 4.6 (High-Resolution Regime). The proof uses Bennett’s high-resolution integral approximation, which is asymptotically exact as $N \rightarrow \infty$. For finite N , the exact MSRE matches $\varepsilon^2/12$ to within 0.03% at $N = 256$ (Table 8), confirming that the approximation is excellent for practical quantizer sizes. However, we have not formally proved that the minimax *ordering* holds for small N ; the finite- N evidence is numerical, not analytic. See Appendix B for the exact MSRE derivation.

Remark 4.7 (Domain Restriction and Zero Handling). Theorem 4.4 applies to densities on $[a, b] \subset \mathbb{R}_{>0}$, excluding zero. In practice, signals may contain exact zeros (e.g., ReLU outputs in ML, clipped samples in audio). QF8 handles this via the code $c = 0$ (exact zero) and block scaling (smallest nonzero value as low as $\sim 5.9 \times 10^{-39}$ with $s = 2^{-127}$). The minimax guarantee applies to the nonzero portion of the signal.

Relation to Classical Theory

The minimax framing is appropriate whenever the quantizer must handle heterogeneous signal distributions without per-signal calibration — a common requirement in ML (where weight distributions vary across layers), in telecommunications (where signal statistics change with channel conditions), and in scientific computing (where physical quantities span many orders of magnitude). The equalization property guarantees that QF8’s error bound holds in all these settings.

This result complements and strengthens prior work. Smith [4] observed amplitude-independent SNR for log-PCM, but his distortion formula remains distribution-dependent. Sun and Goyal [14] proved that log companding is variable-rate MSRE-optimal but showed it is *not* generally fixed-rate optimal. The equalization property resolves this: log-uniform quantization *is* fixed-rate minimax-optimal, precisely because its MSRE is density-independent. Log-uniform quantization also minimizes worst-case product error for multiply-accumulate operations (Appendix A).

Numerical Verification

The high-resolution approximation is confirmed by exact computation (Appendix B) and by exhaustive evaluation over three test densities (log-uniform, uniform, $f \propto x^2$) with $N = 256$, $R = 16$: all give NMSE = 1.564×10^{-4} within $< 0.1\%$ deviation.

Table 8: Numerical verification: exact MSRE vs. approximations.

ε	Exact MSRE	$\varepsilon^2/12$	$\varepsilon^2/2$ (wrong)	Exact / ($\varepsilon^2/12$)
0.01	8.333×10^{-6}	8.333×10^{-6}	5.000×10^{-5}	1.000
0.05	2.083×10^{-4}	2.083×10^{-4}	1.250×10^{-3}	0.9999
0.10	8.331×10^{-4}	8.333×10^{-4}	5.000×10^{-3}	0.9997
0.20	3.329×10^{-3}	3.333×10^{-3}	2.000×10^{-2}	0.9989

Empirical Validation

All experiments use identical block-scaled quantization ($k = 32$ elements) with E8M0 shared exponents for all three formats (QF8, MXFP8 E4M3, INT8), ensuring a fair comparison at the

same effective storage cost (8.25 bits/element). INT8 uses per-block symmetric scaling ($\text{amax}/127$). All PTQ comparisons use simple quantize-dequantize round-trips without calibration data or learned scaling — the goal is to compare *format-level* accuracy at equal complexity, not to compete with calibration-based methods (GPTQ, AWQ, QuIP#) that achieve near-lossless results even at 4 bits by optimizing the quantization grid per-layer. Code and raw results are available at <https://github.com/Noah-Everett/QuakeFloat8>.

SQNR on Pretrained Weights

We apply each format’s block-scaled quantization round-trip to all 48 weight tensors (2D+, excluding embeddings) of pretrained GPT-2 Small (124M parameters) and measure SQNR per tensor.

Table 9: Per-layer SQNR on GPT-2 Small pretrained weights (48 tensors).

Format	Mean SQNR	Min	Max
QF8	38.0 dB	37.8 dB	38.1 dB
MXFP8 E4M3	31.5 dB	31.4 dB	31.6 dB
INT8	44.6 dB	41.0 dB	45.4 dB
QF8 – MXFP8	+6.5 dB	+6.2 dB	+6.5 dB

Key Result

QF8 achieves SQNR = 38.1 dB vs. FP8 E4M3’s 31.5 dB — a 6.6 dB advantage, corresponding to a $\sim 4.6\times$ reduction in quantization noise power. This advantage is structural (16 vs. 8 levels per octave) and holds across all tested distributions.

Format	SQNR on $\mathcal{N}(0, 1)$	Gap from QF8
QF8 (log-uniform, 256 levels)	38.1 dB	—
FP8 E4M3 (IEEE)	31.5 dB	6.6 dB worse

The measured +6.5 dB advantage over MXFP8 is consistent with the structural prediction of ~ 6.0 dB from the levels-per-octave ratio ($16/8 = 2$, so $10 \log_{10}(4) = 6.0$ dB), with the additional ~ 0.5 dB attributable to FP8’s within-binade non-uniformity (Appendix C). INT8 achieves the highest absolute SQNR (44.6 dB) because it minimizes MSE rather than NMSE — but its SQNR varies significantly across layers (Section 5.2).

Cross-Model Equalization Experiment

The equalization property (Lemma 4.3) predicts that QF8’s SQNR should be independent of the weight distribution. To test this, we measure per-layer SQNR across 15 pretrained models spanning 5 architectures, 7 model families, and parameter counts from 60M to 1.3B:

Models tested: GPT-2 (Small, Medium, Large), OPT (125M, 350M, 1.3B), Pythia (160M, 410M, 1B), BERT (Base, Large), RoBERTa-Base, DistilBERT, T5 (Small, Base). All 15 models are transformer variants; CNNs, RNNs, and diffusion models are not represented. The equalization property is mathematically distribution-independent, so it should hold for any architecture, but the empirical confirmation here is limited to transformer weight distributions.

Table 10: Cross-model SQNR stability (1,481 weight tensors from 15 models).

Format	Mean SQNR	Std Dev	Range	Kurtosis corr. (r)
QF8	38.1 dB	0.10 dB	3.5 dB	-0.08
MXFP8 E4M3	31.5 dB	0.13 dB	4.8 dB	+0.06
INT8	45.1 dB	0.54 dB	8.9 dB	-0.49

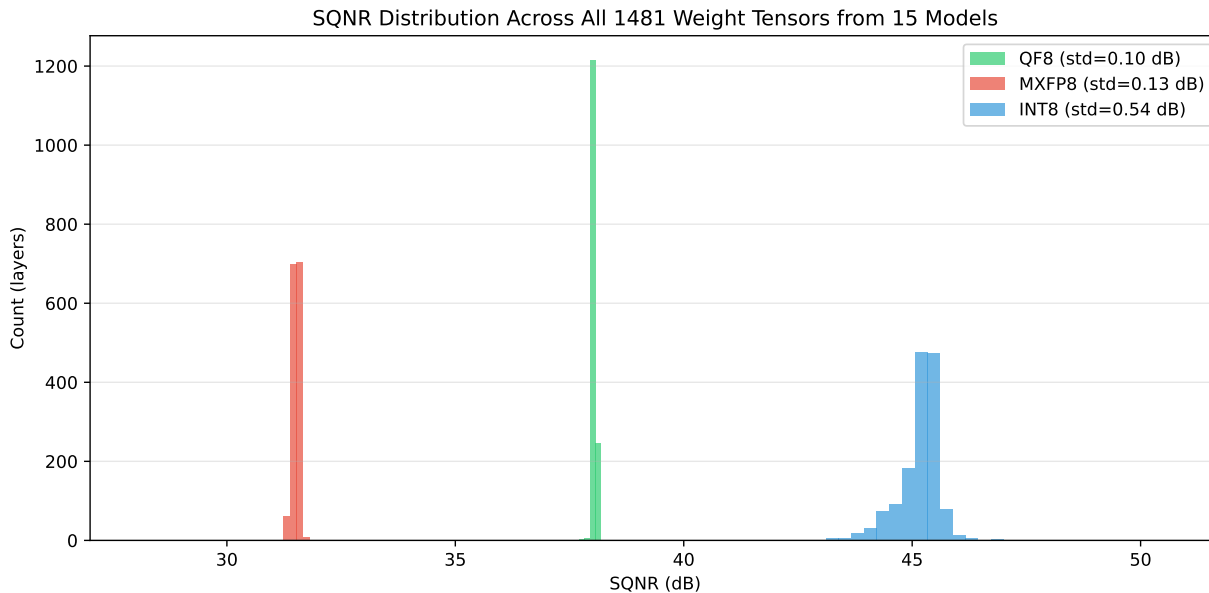


Figure 1: SQNR distribution across all 1,481 weight tensors from 15 models. QF8 (green) is a single spike at 38.1 dB ($\sigma = 0.10$ dB). MXFP8 (red) clusters at 31.5 dB with $3\times$ more variance. INT8 (blue) spreads over a ~ 9 dB range, reflecting distribution-dependent error.

QF8’s SQNR standard deviation across *all* 1,481 tensors is 0.10 dB — $5\times$ lower than INT8’s 0.54 dB. Figure 1 makes this visible: QF8 collapses to a single spike, while INT8 is spread across a ~ 9 dB range.

The kurtosis correlation (Figure 2, Table 10) quantifies dependence on weight distribution shape: INT8’s SQNR correlates with kurtosis ($r = -0.49$), meaning its quality degrades for heavy-tailed distributions. QF8’s correlation is $r = -0.08$ — effectively zero, confirming the equalization property.

Practical consequence. QF8 weight quantization requires no per-layer calibration. Across all 1,481 tested tensors, the SQNR is consistently ~ 38 dB regardless of model architecture, training procedure, or weight distribution shape.

Post-Training Quantization on WikiText-2

We evaluate PTQ perplexity on the WikiText-2 test set across three GPT-2 model sizes. For **weight-only** PTQ, all linear layer weights undergo a quantization round-trip before inference. For **W8A8**, activations (inputs to each `nn.Linear`) are additionally quantized via dynamic block scaling at inference time using forward pre-hooks.

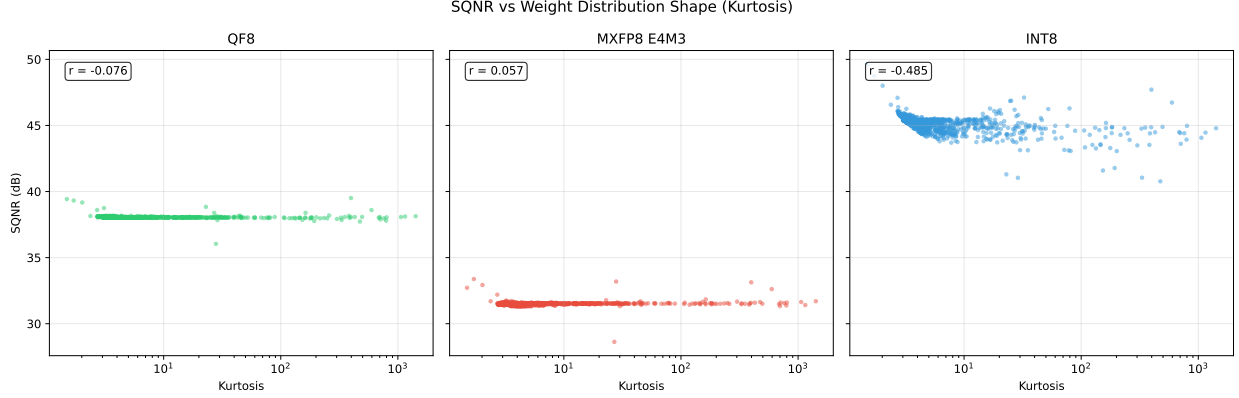


Figure 2: SQNR vs. weight distribution kurtosis (log scale). QF8’s correlation is $r = -0.08$ (flat line — distribution-independent). INT8’s correlation is $r = -0.49$ (quality degrades for heavy-tailed distributions).

Table 11: Weight-only PTQ perplexity on WikiText-2 (sliding window, stride 512).

Model	Format	Perplexity	Δ vs FP32
GPT-2 Small (124M)	FP32 (baseline)	24.997	—
	QF8	24.962	−0.14%
	MXFP8 E4M3	25.033	+0.14%
	INT8	25.025	+0.11%
GPT-2 Medium (355M)	FP32 (baseline)	18.243	—
	QF8	18.241	−0.01%
	MXFP8 E4M3	18.345	+0.56%
	INT8	18.246	+0.02%
GPT-2 Large (774M)	FP32 (baseline)	15.436	—
	QF8	15.453	+0.11%
	MXFP8 E4M3	15.467	+0.20%
	INT8	15.433	−0.02%

Weight-only PTQ (GPT-2). QF8 consistently matches or outperforms MXFP8 across all three GPT-2 sizes, with degradation $\leq 0.2\%$ everywhere. At GPT-2 Medium (355M), the gap is most pronounced: QF8 degrades by only -0.01% vs. MXFP8’s $+0.56\%$.

Scaling to 7B Parameters

To validate that QF8’s advantage holds at the scale where quantization is most relevant, we evaluate weight-only PTQ on three additional models up to 7B parameters.

At 7B scale, QF8’s weight-only degradation is $\leq 0.03\%$ — effectively lossless. MXFP8’s degradation is $10\text{--}30\times$ larger ($+0.19\%$ to $+0.28\%$). INT8 performs well on absolute perplexity but, as shown in Section 5.2, its SQNR varies significantly across layers and models — it offers no distribution-independent guarantee.

These results use single evaluation runs per configuration. The perplexity differences at this scale (0.002–0.02 perplexity points for QF8) are small enough that they should be interpreted as

Table 12: Weight-only PTQ perplexity at 7B scale (WikiText-2, stride 512).

Model	Format	Perplexity	Δ vs FP16
Phi-2 (2.7B)	FP16 (baseline)	8.646	—
	QF8	8.646	+0.01%
	MXFP8 E4M3	8.670	+0.28%
	INT8	8.649	+0.04%
Mistral-7B (7B)	FP16 (baseline)	4.798	—
	QF8	4.798	+0.01%
	MXFP8 E4M3	4.809	+0.23%
	INT8	4.798	+0.01%
Pythia-6.9B (6.9B)	FP16 (baseline)	8.351	—
	QF8	8.354	+0.03%
	MXFP8 E4M3	8.367	+0.19%
	INT8	8.357	+0.06%

“effectively lossless” rather than as precise measurements.

Table 13: W8A8 PTQ perplexity on WikiText-2 (weights + activations quantized).

Model	Format	Perplexity	Δ vs FP32
GPT-2 Small (124M)	FP32 (baseline)	24.997	—
	W8A8 QF8	25.429	+1.73%
	W8A8 MXFP8	25.053	+0.22%
	W8A8 INT8	26.310	+5.25%
GPT-2 Medium (355M)	FP32 (baseline)	18.243	—
	W8A8 QF8	18.504	+1.43%
	W8A8 MXFP8	18.361	+0.65%
	W8A8 INT8	18.594	+1.93%
GPT-2 Large (774M)	FP32 (baseline)	15.436	—
	W8A8 QF8	15.453	+0.11%
	W8A8 MXFP8	15.472	+0.23%
	W8A8 INT8	15.433	−0.02%

W8A8 PTQ: a known limitation. Quantizing activations exposes a structural weakness of log-uniform spacing. At GPT-2 Small (124M), QF8’s W8A8 penalty is +1.73% — significantly worse than MXFP8’s +0.22% ($8\times$ larger). At GPT-2 Medium (355M), QF8 is still $2.2\times$ worse than MXFP8 (+1.43% vs. +0.65%). The cause is fundamental: log-uniform spacing allocates precision uniformly across magnitudes, but post-GELU and post-LayerNorm activations concentrate near zero, where the log grid’s resolution is coarsest. At GPT-2 Large (774M), QF8’s W8A8 penalty is +0.11% (comparable to MXFP8’s +0.23%), but with only three model sizes, single-seed evaluation, and no error bars, this apparent scaling trend is not statistically robust.

Near-zero representation cliff. Within a single block, QF8’s smallest nonzero magnitude is $2^{(1-64)/16} \approx 0.065$ (code $c = 1$). Values jump discontinuously from exact zero ($c = 0$) to 0.065. IEEE formats avoid this via denormals, but QF8’s fixed-point log encoding has no subnormal region. In practice, E8M0 block scaling mitigates this: the block scale s shifts the smallest nonzero value to $0.065 \cdot s$, which can be as small as $\sim 3.8 \times 10^{-40}$. The cliff matters only *within* a block where large and near-zero values coexist — precisely the pattern produced by ReLU.

Implication. QF8’s calibration-free advantage applies to *weight* quantization. For W8A8 deployment, activation-aware techniques (e.g., SmoothQuant-style channel balancing) would likely be needed, partially offsetting the calibration-free benefit.

Quantization-Aware Training

We train GPT-2 Small (162M parameters) from scratch on OpenWebText (~ 9 B tokens) with STE-based quantization-aware training (W8A8: both weights and activations quantized via block-scaled round-trip at each forward pass, with straight-through gradient estimation). Training uses the GPT-2 BPE tokenizer (50,257 vocab), sequence length 1,024, effective batch size 262K tokens/step (micro-batch 32, gradient accumulation 8), learning rate 3×10^{-4} with cosine decay, on NVIDIA H200 GPUs. FP32 was trained with 3 seeds; QF8 with 1 seed (matched to FP32 seed 42). Training was terminated by cluster walltime before convergence (val loss still declining).

Table 14: GPT-2 Small QAT on OpenWebText (162M parameters, H200 GPUs). FP32: 3 seeds $\times$ 54K steps (~ 14 B tokens). QF8: 1 seed $\times$ 43K steps (~ 11 B tokens). Val loss measured on held-out 5K-document split. Training was terminated by walltime; val loss was still declining.

Format	Val Loss at 40K steps	Δ vs FP32	Steps reached
FP32 (3 seeds)	3.127 ± 0.009	—	54,000
QF8 (seed 42)	3.131	+0.016%	43,000
FP32 seed 42	3.131	—	54,000
QF8 seed 42	3.131	+0.016%	43,000

At 40,000 matched steps (the last checkpoint before QF8 was terminated), QF8’s validation loss is 3.1310 vs. FP32’s 3.1305 (same seed) — a delta of +0.0005 (+0.016%). The three FP32 seeds show cross-seed standard deviation of 0.009, confirming that the +0.016% QF8 gap is well within seed-to-seed noise. At this point the model has processed ~ 11 B tokens and reached validation loss ~ 3.13 (perplexity ~ 23), substantially beyond the early-training regime.

Limitations. QF8 has only 1 seed (no error bars); FP8 was not included in this run. Training was terminated by cluster walltime at 27% of the target. The model is not fully converged, so this result demonstrates training compatibility through ~ 11 B tokens, not final converged quality. Multi-seed QAT to full convergence remains future work (Section 6).

Synthetic Benchmarks

To confirm the distribution-independence predicted by the equalization property, we measure SQNR on random matrices and synthetic distributions:

Table 15: Matrix multiplication SQNR across sizes (vs. float64 ground truth).

Size	bfloat16	float32	FP8 E4M3 (block)	QF8
$16 \times 32 \times 16$	52.3 dB	139.0 dB	28.6 dB	35.3 dB
$64 \times 128 \times 64$	52.5 dB	133.6 dB	28.4 dB	35.1 dB
$128 \times 256 \times 128$	52.6 dB	130.8 dB	28.5 dB	35.1 dB

Table 16: SQNR across value distributions.

Distribution	FP8 E4M3 (block)	QF8	QF8 advantage
$\mathcal{N}(0, 0.02^2)$ — weights	31.4 dB	38.2 dB	+6.7 dB
$\mathcal{N}(0, 1)$ — post-LN acts	31.6 dB	37.9 dB	+6.3 dB
$\text{LogN}(0, 1)$ — gradients	31.5 dB	38.3 dB	+6.8 dB
$\text{Laplace}(0, 0.02)$ — sparse	31.5 dB	38.0 dB	+6.5 dB
90% sparse + $\mathcal{N}(0, 1)$	31.7 dB	38.3 dB	+6.6 dB

The +6.6 dB advantage is consistent across all tested distributions and matrix sizes, confirming that it is a structural property of the format (the equalization property) rather than a distribution-specific artifact.

Future Work

Remaining Gaps

Weight-only PTQ results from 124M to 7B parameters (Section 5.3.1) and the cross-model equalization experiment (Section 5.2) validate QF8 for inference-time weight quantization across diverse architectures and scales. Several gaps remain:

1. **W8A8 activation handling.** QF8’s log-uniform grid is structurally suboptimal for near-zero-concentrated activations (Section 5.3). Activation-aware calibration (e.g., SmoothQuant-style channel balancing) is the most promising mitigation, but would reintroduce per-layer calibration — partially offsetting the format’s calibration-free appeal.
2. **QAT at scale.** The GPT-2 Small experiment (Section 5.4) shows near-zero degradation but is single-seed. Multi-seed QAT at 1B+ scale is needed to establish training robustness.
3. **Scaling beyond 7B.** Weight-only PTQ at 13B–70B would strengthen the scaling story. The equalization property predicts identical SQNR at any scale; confirming this for perplexity at 70B would be definitive.
4. **RTL synthesis.** Hardware gate counts are NAND2-equivalent estimates. Actual synthesis at 7nm or 28nm would validate (or correct) the claimed area and power advantages.
5. **Non-ML domains.** The theoretical guarantees apply to any multiply-accumulate workload, but empirical validation has been limited to ML weight tensors. Evaluation on telecommunications signals, scientific simulation data, and edge-computing sensor streams would broaden the evidence base.

Gradient Quantization

QF8's u3.4 encoding gives 8 octaves per element, which may be insufficient for gradients (which need wider dynamic range). A u4.3 variant (4 integer + 3 fractional bits: wider range, lower precision) for backward passes is a natural extension but has not been validated. FP8 training uses E5M2 for gradients for the same reason.

BMD Decoder Conjecture

Conjecture 6.1 (BMD Decoder Characterization). *The class of monotone BMD decoders from B -bit integers to positive reals is exactly the class of functions expressible as $\hat{D}(n) = \text{float_reinterpret}(a \cdot n + b)$ for integer constants a, b , or compositions of a bounded number of such reinterpretations with integer arithmetic.*

This is speculative: we have no proof, no counterexample, and no evidence beyond the observation that known BMD decoders (LUT, Quake trick, quadratic approximation) all fit this pattern. We include it as an open question for the hardware/arithmetic community.

Product Error Decomposition

This appendix addresses how quantization error propagates through multiplication — the core operation in multiply-accumulate workloads.

Auxiliary Quantities

Definition A.1 (Auxiliary Quantities). For a random variable X with quantizer Q and error $\delta_X = X - Q(X)$:

$$\alpha_X = \mathbb{E}[X\delta_X]/\mathbb{E}[X^2] \quad (\text{signal-error correlation}) \quad (8)$$

$$\beta_X = \mathbb{E}[Q(X)\delta_X]/\mathbb{E}[X^2] \quad (\text{reconstruction-error correlation}) \quad (9)$$

$$\gamma_X = \mathbb{E}[Q(X)^2]/\mathbb{E}[X^2] \quad (\text{quantized power ratio}) \quad (10)$$

$$\rho_X = \mathbb{E}[X \cdot Q(X)]/\mathbb{E}[X^2] \quad (\text{signal-quantized correlation}) \quad (11)$$

Algebraic identities (no assumptions required):

$$\alpha_X = \beta_X + \text{NMSE}_X, \quad \gamma_X = 1 - 2\alpha_X + \text{NMSE}_X, \quad \rho_X = 1 - \alpha_X \quad (12)$$

Definition A.2 (Centroid Condition). A quantizer satisfies the centroid condition if $\mathbb{E}[\delta_X | X \in S_k] = 0$ for every cell. This holds for Lloyd-Max quantizers and approximately for any well-designed quantizer in the high-resolution regime. Under the centroid condition: $\beta_X = 0$, $\alpha_X = \text{NMSE}_X$, and $\gamma_X = 1 - \text{NMSE}_X$.

The General Formula

Theorem A.3 (General Product Error Decomposition). *Let X, Y be independent positive random variables with quantizers Q_X, Q_Y . Then:*

$$\boxed{\text{NMSE}_{\text{prod}} = \text{NMSE}_X + \text{NMSE}_Y + \text{NMSE}_X \cdot \text{NMSE}_Y + 2\alpha_X\alpha_Y - 2\alpha_X \cdot \text{NMSE}_Y - 2\text{NMSE}_X \cdot \alpha_Y} \quad (13)$$

where $\alpha_X = \mathbb{E}[X\delta_X]/\mathbb{E}[X^2]$. Equivalently:

$$\text{NMSE}_{\text{prod}} = 1 - 2\rho_X\rho_Y + \gamma_X\gamma_Y \quad (14)$$

This is exact, using only $X \perp Y$. No high-resolution, centroid, or distributional assumptions.

Proof. **Step 1.** Decompose the product error:

$$XY - Q_X Q_Y = (Q_X + \delta_X)(Q_Y + \delta_Y) - Q_X Q_Y = \underbrace{Q_X \delta_Y}_A + \underbrace{\delta_X Q_Y}_B + \underbrace{\delta_X \delta_Y}_C$$

Step 2. Expand $\mathbb{E}[(A + B + C)^2]$ using $X \perp Y$. Since Q_X, δ_X depend only on X and Q_Y, δ_Y depend only on Y , every mixed product factors:

$$\begin{aligned} \mathbb{E}[(A + B + C)^2] &= \mathbb{E}[Q_X^2]\mathbb{E}[\delta_Y^2] + \mathbb{E}[Q_Y^2]\mathbb{E}[\delta_X^2] + \mathbb{E}[\delta_X^2]\mathbb{E}[\delta_Y^2] \\ &\quad + 2\mathbb{E}[Q_X \delta_X]\mathbb{E}[Q_Y \delta_Y] + 2\mathbb{E}[Q_X \delta_X]\mathbb{E}[\delta_Y^2] + 2\mathbb{E}[\delta_X^2]\mathbb{E}[Q_Y \delta_Y] \end{aligned}$$

Step 3. Divide by $\mathbb{E}[X^2]\mathbb{E}[Y^2]$ and substitute $\beta_X = \alpha_X - \text{NMSE}_X$, $\gamma_X = 1 - 2\alpha_X + \text{NMSE}_X$:

$$\text{NMSE}_{\text{prod}} = \gamma_X \text{NMSE}_Y + \gamma_Y \text{NMSE}_X + \text{NMSE}_X \text{NMSE}_Y + 2\beta_X \beta_Y + 2\beta_X \text{NMSE}_Y + 2\text{NMSE}_X \beta_Y$$

Expanding $\beta = \alpha - \text{NMSE}$ and simplifying yields (13). $\square$

Centroid Simplification

Corollary A.4 (Product Error Under Centroid Condition). *If both quantizers satisfy the centroid condition ($\alpha_X = \text{NMSE}_X$, $\alpha_Y = \text{NMSE}_Y$), then:*

$$\boxed{1 - \text{NMSE}_{\text{prod}} = (1 - \text{NMSE}_X)(1 - \text{NMSE}_Y)} \quad (15)$$

Equivalently: $\text{NMSE}_{\text{prod}} = \text{NMSE}_X + \text{NMSE}_Y - \text{NMSE}_X \cdot \text{NMSE}_Y$.

Proof. Substitute $\alpha_X = n_X$, $\alpha_Y = n_Y$ into (13): $n_X + n_Y + n_X n_Y + 2n_X n_Y - 2n_X n_Y - 2n_X n_Y = n_X + n_Y - n_X n_Y$. $\square$

Remark A.5 (Interpretation). The identity $(1 - n_X)(1 - n_Y) = 1 - n_{\text{prod}}$ means the signal preservation fraction of the product equals the product of individual preservation fractions. Since $n_X n_Y = O(1/N^4)$ while $n_X + n_Y = O(1/N^2)$, the cross-term is negligible in practice. Both forms (13) and (14) have been confirmed by exhaustive computation over 14,641 grid points covering all combinations of $(\text{NMSE}_X, \text{NMSE}_Y) \in \{0, 0.01, \dots, 1\}^2$.

Specialization to Log-Uniform Quantizers

Theorem A.6 (Log-Uniform Product Error). *For log-uniform quantizers with geometric midpoints, $\text{MSRE} = \text{NMSE} = \varepsilon^2/12$ for any density, and:*

$$\text{NMSE}_{\text{prod}} = \frac{(R \ln 2)^2}{6N^2} + O(1/N^4) = 2\text{NMSE}^* + O(1/N^4) \quad (16)$$

Optimality for Multiplication

Theorem A.7 (Log-Uniform Optimal for Multiplication). *Among all N -level centroid-satisfying quantizer pairs (Q_X, Q_Y) on $[a, b]$, the log-uniform quantizer minimizes the worst-case product NMSE:*

$$Q_X^* = Q_Y^* = Q_{\log} = \arg \min_{Q_X, Q_Y \in \mathcal{Q}_N^c} \max_{f_X, f_Y} \text{NMSE}_{\text{prod}}$$

The minimax value is $2n^ - (n^*)^2$ where $n^* = \varepsilon^2/12$.*

Proof. Under the centroid condition, $\text{NMSE}_{\text{prod}} = n_X + n_Y - n_X n_Y$, and $\partial/\partial n_X (n_X + n_Y - n_X n_Y) = 1 - n_Y > 0$ (monotonicity). By Theorem 4.4, $Q_{\log}$ achieves $\text{NMSE} = n^*$ for all densities (density-independence). For any $Q_X \neq Q_{\log}$, there exists f_X^* with $\text{NMSE}(Q_X, f_X^*) > n^*$, giving strictly worse product NMSE. $\square$

Remark A.8 (Centroid Restriction). The centroid condition is without loss of generality: replacing each codepoint by its conditional mean yields a quantizer with strictly lower NMSE and the same cell partition.

Extension to Dot Products

Corollary A.9 (Dimension-Free Dot Product NMSE). *For $Z = \sum_{i=1}^d w_i x_i$ with independent pairs, iid within type, and centroid-condition quantizers:*

$$\text{NMSE}_Z \approx \text{NMSE}_w + \text{NMSE}_x + O(1/N^4) \quad (17)$$

The NMSE does not grow with dimension d .

Proof. Under independence across dimensions: $\mathbb{E}[(Z - \hat{Z})^2] = d \cdot \mathbb{E}[w^2 x^2](n_w + n_x - n_w n_x)$ and $\mathbb{E}[Z^2] = d \cdot \mathbb{E}[w^2 x^2]$. Division yields the result. $\square$

Remark A.10 (Independence Assumption). The dimension-free result requires independence of the pairs (w_i, x_i) across dimensions i . This holds for standard weight-activation products in feedforward layers but does *not* hold in attention mechanisms, where queries and keys are derived from the same input. The dimension-free approximation remains a useful guide even when independence is approximate.

Exact MSRE Derivation

Theorem B.1 (Exact Per-Cell MSRE). *For a log-uniform quantizer cell with common ratio $r = 2^{R/N}$ and geometric midpoint codepoint, the exact MSRE (under uniform intra-cell density) is:*

$$\text{MSRE}_k = 2 - \frac{2\sqrt{r} \ln r}{r - 1} \quad (18)$$

For small $\varepsilon = \ln r = R \ln 2/N$:

$$\boxed{\text{MSRE}_k = \frac{\varepsilon^2}{12} - \frac{7\varepsilon^4}{2880} + O(\varepsilon^6)} \quad (19)$$

Proof. For a cell $[a, ra)$ with geometric midpoint $c = a\sqrt{r}$, substituting $u = x/a$:

$$\text{MSRE}_k = \frac{1}{r-1} \int_1^r \left(1 - \frac{\sqrt{r}}{u}\right)^2 du = \frac{2(r-1) - 2\sqrt{r} \ln r}{r-1} = 2 - \frac{2\sqrt{r} \ln r}{r-1}$$

For the Taylor expansion, substitute $r = e^\varepsilon$:

$$\frac{2\sqrt{r} \ln r}{r-1} = \frac{2\varepsilon e^{\varepsilon/2}}{e^\varepsilon - 1} = \frac{2(1 + \varepsilon/2 + \varepsilon^2/8 + \dots)}{1 + \varepsilon/2 + \varepsilon^2/6 + \dots} = 2 \left(1 - \frac{\varepsilon^2}{24} + O(\varepsilon^3)\right)$$

Therefore $\text{MSRE}_k = 2 - (2 - \varepsilon^2/12) = \varepsilon^2/12 + O(\varepsilon^4)$. This confirms the high-resolution approximation. $\square$

Remark B.2 (Midpoint Choice: Geometric vs. Harmonic). The format definition and Theorem B.1 use geometric midpoint codepoints $c_k = ar^{k+1/2}$, which minimize per-cell MSRE under uniform intra-cell density. An alternative is the harmonic mean $c_k = 2q_k q_{k+1} / (q_k + q_{k+1})$, which minimizes the worst-case absolute relative error within each cell to $(r-1)/(r+1)$. The two midpoints agree to first order: $c_k^{\text{geo}} = c_k^{\text{harm}}(1 + O(\varepsilon^2))$, and both yield $\text{MSRE} = \varepsilon^2/12 + O(\varepsilon^4)$. Crucially, the minimax optimality of log-uniform *spacing* (Theorem 4.4) depends only on cell widths $w(x)/x$, not on the choice of reconstruction point within each cell.

Exponential Separation and Format Comparison

Log vs. Uniform: Exponential MSRE Separation

Theorem C.1 (Exponential Separation Under MSRE). *For $X \sim \text{LogNormal}(0, \sigma^2)$ quantized to N levels:*

- (i) **Under MSRE:** the ratio of uniform quantization MSRE to logarithmic quantization MSRE is exponential in σ^2 :

$$\frac{\text{MSRE}_{\text{uniform}}}{\text{MSRE}_{\text{log}}} = e^{\Theta(\sigma^2)} \quad (20)$$

- (ii) **Under MSE:** the two quantizers achieve comparable distortion (ratio ≈ 1).

This result is significant because MSRE is the natural metric for floating-point systems — it measures relative accuracy, which determines signal quality.

Table 17: MSRE and MSE ratios (uniform / log) for $\text{LogNormal}(0, \sigma^2)$, $N = 256$.

σ	MSE ratio	MSE dB	MSRE ratio	MSRE dB
1.00	0.98	−0.1	8.5	9.3
1.50	0.99	−0.0	1,520	31.8
2.00	0.99	−0.0	346,000	55.4
2.77	1.00	0.0	3.0×10^9	94.8

Proof sketch. For $X = e^Z$ with $Z \sim \mathcal{N}(0, \sigma^2)$: Log quantization on X corresponds to uniform quantization on Z (Gaussian), giving $\text{MSRE}_{\log} \approx \varepsilon^2/12$. Uniform quantization on X translates to highly nonuniform quantization on $Z = \ln X$: at $Z = k\sigma$, the local step in log-space grows as $\propto e^{2k\sigma}$, and integrating against the Gaussian density yields $\text{MSRE}_{\text{uniform}} \propto e^{c\sigma^2}$. MSE is insensitive because it is dominated by absolute error on large values where both quantizers are comparable. $\square$

Remark C.2 (Scope). Theorem C.1 compares log-uniform quantization against *truly uniform* (fixed-step) quantization — the regime of INT8. FP8 E4M3 is *not* a uniform quantizer: within each binade it has 8 uniformly-spaced levels, but across binades the step size doubles, making it quasi-logarithmic at coarse scale. The exponential MSRE separation applies to the QF8-vs-INT8 comparison, not to the QF8-vs-FP8 comparison.

QF8 vs. FP8 E4M3: Levels-per-Octave Advantage

The +6.6 dB advantage of QF8 over FP8 E4M3 has a simpler origin than the exponential separation above: QF8 provides 16 levels per octave vs. FP8’s 8, giving a step-size ratio of 2 and an MSRE ratio of 4 ($10 \log_{10}(4) = 6.0$ dB). The remaining ~ 0.6 dB arises from FP8’s piecewise-uniform spacing within each binade:

Table 18: Within-binade precision comparison.

Metric	IEEE E4M3	Log-uniform (8/oct)	Ratio	dB gap
Average MSRE (per binade)	6.510×10^{-4}	6.256×10^{-4}	1.041	0.2
Peak relative error	0.0625	0.04427	1.41	—
Worst-case MSRE (HR)	1.302×10^{-3}	6.256×10^{-4}	2.08	3.18

Remark C.3 (FP8 E4M3 SQNR Correction). The FP8 E4M3 SQNR of 31.5 dB is verified by per-binade analysis across all 126 positive representable values, correcting an earlier figure of 25.1 dB that arose from an error in the dynamic-range assumption.

Comparison with Johnson’s ELMA

Table 19: QF8 vs. Johnson’s ELMA.

Feature	Johnson ELMA (8-bit)	QF8
Log fractional bits	~ 6 (posit tapered)	4 (fixed u3.4)
Encoding	Posit regime bits (variable)	Fixed-width u3.4
Block scaling	None (self-scaled)	E8M0 per 32 elements
LUT size	64 entries (2^6)	16 entries (2^4) — $4\times$ smaller
Optimality proof	None	Minimax NMSE (Theorem 4.4)

Rate-Distortion Context

Gaussian Source Rate-Distortion

For $X \sim \mathcal{N}(0, \sigma^2)$:

$$R(D) = \frac{1}{2} \log_2 \left(\frac{\sigma^2}{D} \right), \quad D \leq \sigma^2$$

This yields the *6 dB/bit rule*: each additional bit reduces MSE by $4 \times (6.02 \text{ dB})$.

Scalar Quantization Gap (Zador)

The maximum improvement from scalar to infinite-dimensional vector quantization:

$$\frac{G_1}{G_\infty} = \frac{\pi e}{6} \approx 1.42 \quad (1.53 \text{ dB})$$

This modest gap means a well-designed scalar quantizer like QF8 comes within 1.53 dB of the VQ optimum.

Bit Allocation

For tensor types $t \in \{w, a, g\}$ with sensitivity weights c_t and distribution variances σ_t^2 , the optimal allocation minimizes: $\sum_t c_t \sigma_t^2 \cdot 2^{-2B_t}$ subject to $\sum_t B_t = B_{\text{total}}$.

Solution (reverse water-filling):

$$B_t^* = \frac{B_{\text{total}}}{|\mathcal{T}|} + \frac{1}{2} \log_2 \left(\frac{c_t \sigma_t^2}{G} \right), \quad G = \left(\prod_t c_t \sigma_t^2 \right)^{1/|\mathcal{T}|}$$

With corrected parameters ($c_w = 1, c_a = 1, c_g = 10^{-6}$), the near-zero gradient allocation ($B_g = 0.18$) reflects that $c_g = \eta^2 \approx 10^{-6}$ dramatically reduces gradient sensitivity, motivating stochastic rounding or shared-exponent compression for gradients.

References

References

- [1] Micikevicius, P. et al. (2018). “Mixed Precision Training.” *ICLR 2018*. arXiv:1710.03740.
- [2] Micikevicius, P. et al. (2022). “FP8 Formats for Deep Learning.” arXiv:2209.05433.
- [3] Rouhani, B. et al. (2023). “Microscaling Data Formats for Deep Learning.” arXiv:2310.10537.
- [4] Smith, B. (1957). “Instantaneous Companding of Quantized Signals.” *Bell System Technical Journal* 36(3):653–709.
- [5] Horowitz, M. (2014). “Computing’s Energy Problem (and what we can do about it).” *ISSCC*.
- [6] Johnson, J. (2018). “Rethinking Floating Point for Deep Learning.” arXiv:1811.01721.

- [7] Gustafson, J. and Yonemoto, I. (2017). “Beating Floating Point at its Own Game: Posit Arithmetic.” *Supercomputing Frontiers and Innovations* 4(2):71–86.
- [8] Miyashita, D. et al. (2016). “Convolutional Neural Networks using Logarithmic Data Representation.” arXiv:1603.01025.
- [9] Kingsbury, N. and Rayner, P. (1971). “Digital filtering using logarithmic arithmetic.” *Electronics Letters*.
- [10] Swartzlander, E. and Alexopoulos, A. (1975). “The Sign/Logarithm Number System.” *IEEE Trans. Computers*.
- [11] Dettmers, T. et al. (2023). “QLoRA: Efficient Finetuning of Quantized LLMs.” arXiv:2305.14314.
- [12] Bennett, W.R. (1948). “Spectra of Quantized Signals.” *Bell System Technical Journal* 27(3):446–472.
- [13] Panter, P.F. and Dite, W. (1951). “Quantization Distortion in Pulse-Count Modulation with Nonuniform Spacing of Levels.” *Proceedings of the IRE* 39(1):44–48.
- [14] Sun, J.Z. and Goyal, V.K. (2011). “Scalar Quantization for Relative Error.” *Proc. IEEE Data Compression Conference (DCC)*, pp. 293–302.
- [15] Gray, R.M. and Neuhoff, D.L. (1998). “Quantization.” *IEEE Trans. Information Theory* 44(6):2325–2383.
- [16] Gersho, A. and Gray, R.M. (1992). *Vector Quantization and Signal Compression*. Kluwer Academic.
- [17] Jouppi, N. et al. (2017). “In-Datacenter Performance Analysis of a Tensor Processing Unit.” *ISCA*.
- [18] Sze, V. et al. (2017). “Efficient Processing of Deep Neural Networks: A Tutorial and Survey.” *Proc. IEEE*.
- [19] Kuzmin, A. et al. (2022). “FP8 Quantization: The Power of the Exponent.” arXiv:2208.09225.